Sync vs Async Communion

22 April 2026 18:30

Sync vs Async Communication in Microservices

🧠 Core Idea

In microservices, services communicate either:

- **Synchronous (Sync):** Immediate request-response
- **Asynchronous (Async):** Fire-and-forget via events/messages

Choosing correctly impacts scalability, latency, and system resilience.

🔄 Synchronous Communication

📌 What it is

A service calls another service and **waits for response**.

⚙️ Examples

- REST APIs
- gRPC calls

⏰ When to use Sync

Use synchronous communication when:

- Immediate response is required
- User-facing request/response flow
- Simple workflows
- Strong consistency is needed

🎯 Typical use cases

- Login authentication
- Payment confirmation check
- Fetching user profile
- Real-time validation (e.g., inventory check)

⚠️ Downsides

- Tight coupling between services
- Higher latency (blocking calls)
- Cascading failures risk
- Harder to scale under heavy load

⚡ Asynchronous Communication

📌 What it is

Services communicate via **messages/events without waiting for response**.

⚙️ Examples

- Kafka

- RabbitMQ
- Event streaming systems

⏰ When to use Async

Use asynchronous communication when:

- Processing can be delayed
- High scalability is required
- Loose coupling is needed
- Event-driven workflows exist

🎯 Typical use cases

- Order processing pipelines
- Email/SMS notifications
- Payment processing workflows
- Logging and audit events
- Data synchronization between services

⚠️ Downsides

- Eventual consistency
- Harder debugging
- Requires idempotency handling
- Complex event tracking and retries

⚖️ How to Decide (Decision Framework)

🧩 Step 1: Is immediate response required?

- YES → Sync
- NO → Async

🧩 Step 2: Is it user-facing critical path?

- YES → Sync
- NO → Async

🧩 Step 3: Can processing be delayed?

- YES → Async
- NO → Sync

🧩 Step 4: Do we expect high load/scalability needs?

- YES → Async preferred
- NO → Sync acceptable

🧩 Step 5: Is strong consistency required?

- YES → Sync
- NO → Async

🔥 Real-world Hybrid Example

Order Flow:

1. User places order → **Sync call** (validate order)
2. Order service creates order → **Async event**

3. Payment service processes → Async
4. Notification service sends SMS/email → Async

👉 Best systems are usually **hybrid**, not purely sync or async.

🧠 Key Trade-offs

Factor	Sync	Async
Latency	Low (immediate)	Higher (event delay)
Coupling	Tight	Loose
Scalability	Limited	High
Consistency	Strong	Eventual
Complexity	Simple	Complex

💡 Interview Key Lines

- "Use synchronous communication for real-time user-driven flows."
- "Use asynchronous communication for scalability and decoupling."
- "Modern microservices systems are hybrid, balancing both patterns."

🧠 Mental Model

- Sync = phone call ☎ (wait for reply)
- Async = message/postbox 📬 (no waiting)

🚀 One-line Summary

Synchronous communication is used for immediate, user-critical operations, while asynchronous communication is preferred for scalable, decoupled, and event-driven processing where immediate response is not required.

Data decomposition

22 April 2026 18:32

Data Decomposition in Microservices — Quick Revision Flashcards

Basics

Q: What is data decomposition in microservices?
A: Splitting a single shared data model into multiple independent, service-owned data models aligned with business domains.

Q: Core principle of data decomposition?
A: Database per service — each microservice owns its own database.

Q: Why is shared database avoided?
A: It creates tight coupling, limits scalability, and prevents independent deployment.

Design Concepts

Q: On what basis is data decomposed?
A: Based on business domains (DDD - Domain Driven Design).

Q: What is data ownership in microservices?
A: Each service is the single source of truth for its own data.

Q: How do services communicate if they don't share DB?
A: Via APIs (sync) or events (async).

Example Mapping

Q: Example of Order Service data?
A: order_id, items, order_status

Q: Example of Customer Service data?
A: customer_id, name, address

Q: Example of Payment Service data?
A: payment_id, order_id, payment_status

Q: Example of Shipping Service data?
A: shipment_id, tracking_status

Communication

Q: Types of service communication?
A: Synchronous (REST/gRPC) and Asynchronous (Kafka/events)

Q: Example of event-driven flow?
A: OrderPlaced event → triggers Payment Service

Challenges

Q: Key challenge of data decomposition?
A: Eventual consistency instead of strong consistency

Q: Why are distributed transactions hard?
A: No single database manages all services

Q: What issue arises in reporting?
A: Data is distributed, so analytics requires separate pipelines

🧠 Solutions

Q: Pattern used for distributed transactions?
A: Saga pattern

Q: What is CQRS?
A: Separates read and write models for scalability

Q: Why use event sourcing/event-driven architecture?
A: To maintain consistency across distributed services

💡 Interview Key Points

Q: One-liner for data decomposition?
A: Each microservice owns its data and schema independently.

Q: Why avoid shared DB in microservices?
A: To reduce coupling and enable independent scaling and deployment.

Q: How is consistency handled?
A: Through events and eventual consistency, not transactions.

🧠 Mental Model

Q: Monolith vs Microservices analogy?
A: Monolith = one shared brain 🧠 | Microservices = multiple independent brains 🧠🧠🧠

🚀 Final Summary

Q: What is data decomposition in one line?
A: It is the practice of splitting data ownership across services to achieve independent scalability, deployment, and evolution using APIs and events instead of shared databases.

Circuit breaker + retry + fallback patterns

22 April 2026 18:34

Circuit Breaker, Retry & Fallback Patterns (Microservices)

🧠 Core Idea

In distributed microservices, failures are normal, not exceptions. To build resilient systems, we use:

- **Retry** → Try again when failure is temporary
- **Circuit Breaker** → Stop calling failing services
- **Fallback** → Provide alternative response when service fails

🔁 1. Retry Pattern

📌 What it is

Automatically re-attempting a failed request assuming the failure is temporary.

⏰ When to use Retry

- Network glitches
- Temporary service overload
- Timeout errors
- Transient database issues

⚙️ How it works

1. Call service
2. If failure occurs → retry
3. Stop after max attempts

⚠️ Risks

- Can increase load on already failing system
- May worsen outages if misused
- Requires backoff strategy

🧪 Best Practices

- Use **exponential backoff**
- Add **jitter** (random delay)
- Limit retry count
- Retry only idempotent operations

⚡ 2. Circuit Breaker Pattern

📌 What it is

Prevents repeated calls to a failing service by "opening the circuit" after failures.

🔌 States

Closed
- Normal operation
- Requests pass through

Open
- Service is failing
- Requests are blocked immediately

Half-Open
- Testing recovery
- Limited requests allowed

When to use Circuit Breaker
- High dependency services
- External APIs
- Payment gateways
- Downstream microservices prone to failure

Benefits
- Prevents cascading failures
- Reduces load on failing services
- Improves system stability

Risks
- Incorrect thresholds may block healthy services
- Requires tuning

3. Fallback Pattern

What it is
Providing an alternative response when the primary service fails.

When to use Fallback
- Non-critical features
- Read-heavy systems
- Degraded service acceptable scenarios

Examples
- Cached data instead of live API
- Default response message
- Secondary service provider
- "Service temporarily unavailable" response

Benefits
- Better user experience
- System remains partially functional

Risks
- Stale data
- Inconsistent user experience
- Can hide real issues if overused

How They Work Together

Typical flow:

1. Client calls Service A
2. Service A calls Service B
3. If failure:
    - Retry first
    - If still failing → Circuit Breaker may open
    - If circuit open → Fallback response used

Comparison Table

Pattern	Purpose	Trigger	Outcome
Retry	Recover from transient failures	Temporary error	Re-attempt request
Circuit Breaker	Prevent system overload	Repeated failures	Block requests
Fallback	Maintain functionality	Service unavailable	Alternative response

Key Interview Insights

- Retry alone can amplify load during outages
- Circuit breaker prevents failure propagation
- Fallback ensures graceful degradation
- All three are often used together in production systems

Real-world Example (E-commerce)

Payment Flow:

- Retry: Retry payment gateway call on timeout
- Circuit Breaker: Stop calling gateway if failures spike
- Fallback: Mark order as "pending payment" and notify user

Mental Model

- Retry = knock again on door
- Circuit Breaker = stop knocking, door might be broken
- Fallback = leave a note and walk away

One-line Summary

Retry handles temporary failures, circuit breaker prevents system overload from persistent failures, and fallback ensures graceful degradation when services are unavailable.

Timeout vs Retry vs Circuit Breaker — Decision Tree (Microservices)

🧠 Core Idea

In distributed systems, failures are expected. These three patterns work together to prevent cascading failures and improve resilience:

- **Timeout** → Don't wait forever
- **Retry** → Try again if failure is temporary
- **Circuit Breaker** → Stop calling a failing service

⚡ 1. TIMEOUT — First Line of Defense

📌 What it does

Limits how long a service call can take before it is aborted.

⏰ When to use Timeout

Always. Every remote call should have a timeout.

- REST/gRPC calls
- DB queries
- External APIs

🎯 Why it matters

- Prevents thread blocking
- Avoids resource exhaustion
- Stops infinite waiting

💡 Key Rule

If you don't set timeout, system stability is already compromised.

🔁 2. RETRY — For Temporary Failures

📌 What it does

Re-attempts failed operations assuming failure is transient.

⏰ When to use Retry

Use only when failure is likely temporary:

- Network glitches
- Temporary overload
- 5xx server errors
- Timeout from downstream service

⚠️ When NOT to retry

- 4xx errors (client errors)
- Non-idempotent operations (unless carefully handled)

Best Practices

- Exponential backoff
- Add jitter (random delay)
- Limit retry attempts
- Retry only idempotent operations

3. CIRCUIT BREAKER — System Protection Layer

What it does

Stops calling a failing service to prevent cascading failures.

States

Closed

- Normal operation
- Requests pass through

Open

- Failures exceed threshold
- Requests are blocked immediately

Half-Open

- Limited test requests allowed
- Checks if service recovered

When to use Circuit Breaker

- Downstream services frequently failing
- External dependencies (payment gateways, third-party APIs)
- High traffic systems

DECISION TREE (Interview Gold)

Step 1: Should I wait for response?

- YES → Apply **Timeout**
- NO → Async flow (not covered here)

Step 2: Did the request fail?

- YES → Go to Retry decision
- NO → Done

Step 3: Is failure temporary?

- YES → Apply **Retry**
- NO → Skip retry

Step 4: Are failures frequent/repeated?

- YES → Activate **Circuit Breaker**
- NO → Continue normal retry logic

Step 5: Is service currently unavailable?

- YES → Use **Circuit Breaker + Fallback**
- NO → Proceed normally

🔄 HOW THEY WORK TOGETHER

Typical flow:

1. Client calls Service B
2. Timeout ensures call doesn't hang
3. If failure occurs → Retry logic kicks in
4. If failures persist → Circuit Breaker opens
5. When circuit is open → Fallback response is used

⚖️ QUICK COMPARISON

Pattern	Purpose	Trigger	Effect
Timeout	Limit waiting time	Slow response	Abort request
Retry	Handle transient failures	Temporary error	Re-attempt call
Circuit Breaker	Prevent cascading failures	Repeated failures	Block requests

🧠 INTERVIEW INSIGHT

- Timeout is mandatory for every remote call
- Retry improves reliability but can increase load
- Circuit breaker protects system stability
- Fallback ensures graceful degradation

💡 REAL-WORLD EXAMPLE (E-Commerce)

Payment Service Call:

- Timeout: 2 seconds max wait
- Retry: 2 attempts with backoff
- Circuit Breaker: opens after repeated gateway failures
- Fallback: mark order as "payment pending"

🧠 MENTAL MODEL

- Timeout = alarm clock ⏰ (stop waiting)
- Retry = knock again 🚪 (try once more)
- Circuit breaker = safety switch 🔌 (stop the system)

🚀 ONE-LINE SUMMARY

Timeout prevents indefinite waiting, retry handles temporary failures, and circuit breaker prevents system overload from persistent downstream issues.

Rate limitter

22 April 2026 18:43

Rate Limiter — Quick Explanation (Microservices)

🧠 What is a Rate Limiter?

A **rate limiter** controls the number of requests a client/user/service can make within a given time window.

👉 It protects systems from:

- Overload
- Abuse (DDoS-like spikes)
- Noisy neighbors

⚡ Why it is used

- Prevent backend crashes under high traffic
- Ensure fair usage among users/services
- Maintain system stability and SLA
- Protect expensive downstream dependencies

🧩 Where it is applied

- API Gateway (most common)
- Load balancer
- Application layer (service-level rate limiting)
- CDN / edge layer

🔄 Common Rate Limiting Algorithms

1. Fixed Window

- Counts requests in fixed time buckets (e.g., 100 req/min)

✓ Simple ❌ Can cause burst at window edges

2. Sliding Window

- Smooths request counting over time

✓ More accurate than fixed window ❌ Slightly more complex

3. Token Bucket

- Tokens are added at a fixed rate
- Each request consumes a token

✓ Allows bursts ✓ Widely used in production

4. Leaky Bucket

- Requests are processed at a constant rate
- Excess requests are queued or dropped

✓ Smooth output traffic ❌ Less flexible for bursts

⚖️ Key Trade-offs

Aspect	Benefit	Drawback
Strict limiting	Protects system	May block valid users
Burst allowance	Better UX	Temporary overload risk
Distributed control	Scales well	Requires coordination

🧠 Key Interview Points

- Rate limiter prevents system overload and abuse
- Most systems use **token bucket at API gateway level**
- Can be implemented in-memory or distributed (Redis)
- Works alongside retries, circuit breakers, and timeouts

🔥 Real-world Example

API Gateway:

- 100 requests/sec per user
- Token bucket used to allow short bursts
- Excess requests return HTTP **429 (Too Many Requests)**

💡 Common HTTP Status Code

- **429 Too Many Requests** → Rate limit exceeded

🧠 Mental Model

- Rate limiter = traffic police 🚦
- It decides who goes, who waits, and who stops

🚀 One-line Summary

A rate limiter controls request flow to ensure system stability, fairness, and protection against overload by enforcing limits per user or service over time.

💥 What is Burst in Rate Limiting?

🧠 Definition
Burst refers to a short-term spike in requests that exceeds the normal allowed rate, but is temporarily permitted by the system.

It allows sudden traffic surges without immediately rejecting requests.

📦 Simple Meaning
If a system allows:
> 100 requests/sec

A burst might allow:
> 150–200 requests in a very short time window

As long as the long-term average stays within limits.

⚙ Where Burst Comes From (Token Bucket Model)

Burst handling is most commonly explained using the **Token Bucket algorithm**:

- Tokens are added at a steady rate (e.g., 10/sec)
- Each request consumes 1 token
- Bucket has a maximum capacity (e.g., 50 tokens)

💡 Key idea:
If tokens accumulate during low traffic, they can be used later all at once.

📊 Example

Configuration:
- Rate: 10 requests/sec
- Bucket capacity: 50 tokens

Scenario:

1. Idle period
- No requests → tokens accumulate up to 50

2. Sudden spike
- 50 requests arrive instantly
- All 50 are allowed → this is a burst

3. After burst
- System returns to steady rate (10/sec)

⚡ Why Burst is Allowed

Real-world traffic is not smooth. Bursts happen due to:
- User rapid clicks
- App retries after network reconnect
- Batch jobs or cron triggers
- Flash sales / sudden traffic spikes

Without burst handling:
- Valid users get throttled unfairly
- System becomes too strict and rigid

⚖ Key Concept

Concept	Meaning
Rate limit	Long-term request control
Burst	Short-term flexibility using stored capacity

🧠 Mental Model

- Rate limit = speed limit on highway 🚗
- Burst = temporarily using buffer lanes during traffic spikes 🚨

🚀 One-line Summary

A burst is a temporary spike in traffic allowed by using stored capacity (like tokens) so that systems can handle real-world unpredictable request patterns smoothly.

Service discovery

22 April 2026 18:46

🔎 Service Discovery in Microservices

🤔 What is Service Discovery?
Service discovery is the mechanism that allows microservices to **find and communicate with each other dynamically**, without hardcoding IP addresses or hostnames.

In simple terms:
> Services don't "remember" where others are — they *ask a registry* where to find them.

🧩 Why Service Discovery is needed

In microservices:
- Instances scale up/down frequently
- IP addresses change dynamically
- Services run in containers/pods (Kubernetes, Docker, cloud)

👉 Hardcoding service locations breaks quickly.

So we need a system that:
- Tracks available service instances
- Provides real-time location info
- Handles dynamic scaling

⚙️ How Service Discovery works

🟢 Step 1: Service Registration
When a service starts:
- It registers itself with a **Service Registry**
- Example: IP, port, health status

🟢 Step 2: Service Lookup
When Service A wants Service B:
- It queries the registry
- Gets a list of healthy instances

🟢 Step 3: Communication
- Service A calls Service B using returned address

🏗️ Types of Service Discovery

1. Client-Side Discovery
Client decides which instance to call.

Flow:
1. Service A queries registry
2. Gets list of Service B instances
3. Chooses one (load balancing logic)
4. Calls it directly

Example tools:
- Netflix Eureka
- Ribbon (client-side load balancing)

Example:

📊 Comparison

Feature	Client-Side	Server-Side
Who decides routing	Client	Load Balancer
Complexity	Higher	Lower
Control	More control	Centralized
Examples	Eureka + Ribbon	Kubernetes, AWS LB

🔥 Real-world Example

E-commerce system:

Services:
- Order Service
- Payment Service
- Inventory Service

Without service discovery:
- Hardcoded IP: `http://10.0.1.5:8080`
- Breaks when service restarts

With service discovery:
1. Payment Service registers itself
2. Order Service asks registry:
   - “Where is Payment Service?”
3. Registry returns healthy instances
4. Order Service calls dynamically

⚠️ Key Challenges

- Registry can become a single point of failure (needs clustering)
- Latency in lookup (mitigated with caching)
- Health check accuracy is critical
- Complex in large distributed systems

🧠 Mental Model

- Service Discovery = “Google Maps for microservices” 🗺️
- Registry = directory of all services 📒
- Load Balancer = traffic controller 🚦

🚀 One-line Summary

Service discovery enables microservices to dynamically find and communicate with each other using a registry or load balancer instead of hardcoded service locations.

SAGA

22 April 2026 19:23

📜 Saga Pattern — Key Points Summary (Microservices)

🧠 What is Saga Pattern?

The **Saga Pattern** is a design pattern used to manage **distributed transactions across multiple microservices** without using a single ACID transaction.

👉 It ensures data consistency through a sequence of **local transactions + compensating actions**.

⚙️ Why Saga Pattern is needed

In microservices:

- No shared database
- No global ACID transaction support
- Failures are common in distributed calls

👉 Problem:

How do we maintain consistency across services?

👉 Solution:

Break one big transaction into multiple small transactions + rollback logic

🔄 How Saga Pattern works

A saga is a sequence of steps:

Step 1: Local transaction

Each service performs its own transaction.

Step 2: Event or command triggers next step

Moves flow to next service.

Step 3: If failure occurs

Compensating transactions are executed to undo previous steps.

🧩 Example (Order Flow)

Step flow:

1. Order Service → Create Order
2. Payment Service → Deduct Money
3. Inventory Service → Reserve Stock
4. Shipping Service → Create Shipment

❌ Failure scenario

If Inventory fails after payment:

- Payment must be rolled back (refund)
- Order marked as failed/cancelled

🔁 Types of Saga

1. Choreography (Event-driven)

- No central controller
- Services communicate via events

Flow:

Order Created → Payment Service listens
Payment Success → Inventory Service listens
Inventory Success → Shipping Service listens

✓ Simple ✓ Decentralized ✗ Hard to track flow in complex systems

2. Orchestration (Central Controller)

- Central saga orchestrator controls flow

Flow:

Saga Orchestrator → calls Order Service
→ calls Payment Service
→ calls Inventory Service

✓ Easier to manage ✓ Clear flow ✗ Central dependency

⚠ Key Concepts

Compensating Transaction

- Reverse action for each step
- Example: refund payment, release stock

Local Transaction

- Each service commits independently
- No distributed lock

Event-driven coordination

- Kafka / RabbitMQ commonly used

Saga vs ACID Transaction

Feature	ACID Transaction	Saga Pattern
Scope	Single DB	Multiple services
Consistency	Strong	Eventual
Locking	Yes	No
Scalability	Limited	High
Failure handling	Rollback	Compensating actions

When to use Saga Pattern

Use Saga when:

- Multiple microservices involved
- Distributed transactions required
- High scalability needed
- Eventual consistency is acceptable

Challenges

- Complex failure handling
- Debugging multi-step flows
- Designing correct compensating actions
- Handling partial failures

Interview Key Points

- Saga = distributed transaction management pattern
- Replaces ACID with eventual consistency
- Uses compensating transactions instead of rollback
- Two types: choreography and orchestration

Mental Model

- ACID = single chef cooking in one kitchen
- Saga = multiple chefs in different kitchens coordinating via messages

One-line Summary

The Saga Pattern manages distributed transactions in microservices by breaking them into a sequence of local transactions with compensating actions to ensure eventual consistency.

Saga vs event driven

22 April 2026 19:31

📜 Saga vs Event-Driven Architecture — Key Differences

🧠 Core Idea

Both Saga and Event-Driven Architecture (EDA) use **events for communication**, but they solve **different problems**:

- **Saga Pattern** → manages **distributed transactions + consistency**
- **Event-Driven Architecture** → enables **loose coupling + asynchronous communication between services**

⚙️ What is Saga Pattern?

Saga is a **distributed transaction management pattern**.

👉 It ensures data consistency across multiple microservices using:

- Local transactions
- Events or commands
- Compensating transactions (rollback-like behavior)

⚡ What is Event-Driven Architecture?

EDA is a **communication style where services emit and react to events asynchronously**.

👉 Focus is on:

- Decoupling services
- Asynchronous processing
- Event propagation

🔄 Key Difference in Purpose

Aspect	Saga Pattern	Event-Driven Architecture
Primary goal	Data consistency across services	Loose coupling & async communication
Problem solved	Distributed transaction management	Service interaction model
Focus	Correctness of business workflow	Communication and scalability

🧩 Example Scenario (Order Flow)

📜 Saga Pattern

Used when strict workflow consistency is needed:

1. Create Order
2. Deduct Payment
3. Reserve Inventory
4. If failure → compensate (refund, release stock)

👉 Ensures end-to-end correctness

⚡ Event-Driven Architecture

Same flow, but focus is different:

1. OrderCreated event emitted
2. Payment Service reacts
3. Inventory Service reacts
4. Shipping Service reacts

👉 No strict coordination, just event reactions

🔁 Relationship Between Them

👉 Saga is often implemented using Event-Driven Architecture

But:

- Not all EDA systems are Sagas
- Not all Sagas are pure EDA (orchestration-based saga uses commands)

🧠 Types Comparison

📜 Saga Types

- Choreography (event-driven)
- Orchestration (central controller)

⚡ Event-Driven Styles

- Pub/Sub model
- Event streaming (Kafka)
- Event notification systems

⚖️ Side-by-Side Comparison

Feature	Saga	Event-Driven Architecture
Nature	Pattern	Architecture style
Purpose	Transaction management	Communication model
Consistency	Stronger (eventual + compensations)	Eventual consistency
Coupling	Moderate	Very low
Control flow	Defined workflow	No strict workflow

🚨 Key Interview Insight

- Saga = business workflow correctness across services
- EDA = system design for async communication
- Saga often *uses* EDA, but EDA is broader

🧠 Mental Model

- Saga = choreographed dance 💃 (steps must complete or be undone)
- EDA = crowd reacting to signals 📡 (everyone responds independently)

🚀 One-line Summary

Saga is a pattern for managing distributed transactions, while Event-Driven Architecture is a broader communication style where services interact via events asynchronously.

Cheat sheet

22 April 2026 19:39

⚡ Microservices Ultra Cheat Sheet (1-Page Revision)

🧠 CORE ARCHITECTURE IDEA

Microservices = independently deployable services with:

- Own database (data decomposition)
- Loose coupling
- Async + sync hybrid communication
- Resilience + scalability patterns

🧩 DATA DECOMPOSITION

- Each service owns its DB (no shared DB)
- Based on business domains (DDD)
- Ensures independent scaling + deployment

⚠ Trade-off:

- Eventual consistency
- Data duplication

🔄 COMMUNICATION

⚡ Sync

- REST / gRPC
- Immediate response required
- Use for: login, validation, payments

⚡ Async

- Kafka / RabbitMQ
- Event-driven
- Use for: notifications, workflows, processing

👉 Most systems = HYBRID

🚦 SERVICE DISCOVERY

- Dynamic service lookup
- Avoid hardcoded URLs

Types:

- Client-side (Eureka)
- Server-side (Kubernetes / Load Balancer)

🚦 RATE LIMITING

- Controls request traffic per user/service
- Prevents overload & abuse

Algorithms:

- Token Bucket (best for bursts)
- Fixed Window
- Sliding Window

Status code: 429 Too Many Requests

RELIABILITY PATTERNS

Timeout

- Always set for remote calls
- Prevents hanging requests

Retry

- Handles transient failures
- Use exponential backoff + jitter
- Only for idempotent operations

Circuit Breaker

- Stops calling failing service States:
- Closed → Open → Half-Open

👉 Prevents cascading failures

SAGA PATTERN

- Distributed transaction management
- Replaces ACID with eventual consistency

Types:

- Choreography (event-driven)
- Orchestration (central controller)

Key concept:

- Compensating transactions (rollback logic)

EVENT-DRIVEN ARCHITECTURE

- Services communicate via events
- Loose coupling
- Asynchronous processing backbone

NOT same as Saga (But Saga often uses EDA)

SAGA vs EDA

Saga	**Event-Driven**
Transaction consistency	Communication model
Workflow control	Event propagation
Uses compensations	No built-in rollback

SYNC vs ASYNC

Sync	**Async**
Immediate response	Event-based
Tight coupling	Loose coupling
Lower scalability	High scalability

RESILIENCE STACK

Order of protection:

1. Timeout (stop waiting)

2. Retry (try again)
3. Circuit Breaker (stop calling)
4. Fallback (graceful response)

🧠 SYSTEM DESIGN FLOW (REAL WORLD)

1. API Gateway entry
2. Rate limiting applied
3. Service discovery resolves services
4. Sync call for critical flow
5. Async events for workflows
6. Saga ensures consistency
7. Retry + timeout + circuit breaker ensure resilience

🧩 KEY PATTERN MAP

- Data ownership → decomposition
- Traffic control → rate limiter
- Service lookup → discovery
- Communication → sync/async
- Consistency → saga
- Scalability → async + EDA
- Resilience → retry + timeout + circuit breaker

🧠 MENTAL MODEL

- API Gateway = front door 🚪
- Rate limiter = security guard 🚦
- Services = independent buildings 🏢
- Kafka = postal system 📬
- Circuit breaker = emergency switch 🔌
- Saga = workflow conductor 🎼

🚀 ONE-LINE SUMMARY

Microservices architecture is a layered system combining decomposition, hybrid communication, event-driven design, and resilience patterns to achieve scalability, reliability, and independence at global scale.

Api gateway

22 April 2026 19:46

🔐 Topic 1: API Gateway (Deep Dive)

😵‍💫 What is an API Gateway?

An **API Gateway** is the single entry point into a microservices system that handles all incoming client requests and routes them to appropriate backend services.

👉 Think of it as the **front-door brain of the system**.

🧩 Why API Gateway exists

Without API Gateway:
- Clients call multiple services directly ❌
- Security logic is duplicated ❌
- Tight coupling between client and services ❌

With API Gateway:
- Single entry point ✔
- Centralized control ✔
- Simplified client logic ✔

⚙ Core Responsibilities

🚦 1. Routing
Routes requests to correct microservices.

Example:
- `/orders → Order Service`
- `/payments → Payment Service`

🔐 2. Authentication & Authorization
- Validates JWT / OAuth2 tokens
- Enforces roles and permissions

Example:
- Admin APIs vs User APIs access control

🚧 3. Rate Limiting
- Controls request volume per user/service

- Prevents abuse and overload

(Protects backend from traffic spikes)

🔄 4. Request Aggregation (BFF Pattern)
Combines multiple service responses into one.

Example:
Instead of 3 calls:
- Order Service
- Payment Service
- Shipping Service

👉 Gateway returns a single combined response

⚡ 5. Load Balancing (Indirect)
Distributes traffic across service instances:
- Round robin
- Least connections
- Weighted routing

🧠 6. Edge Caching
- Caches frequent responses at the gateway
- Reduces backend load and latency

📊 7. Observability Layer
Central place for:
- Logs
- Metrics
- Tracing

👉 Because ALL traffic flows through it

🏗️ Request Flow Example

```text
Client
 ↓
API Gateway
 ↓
Authentication (JWT validation)
 ↓
Rate Limiting check
 ↓
Routing → Order Service
 ↓
```

Order Service → Payment Service (internal call)

Consistency models

22 April 2026 19:49

🧠 Topic 2: Consistency Models in Distributed Systems

🧩 What is Consistency?

Consistency defines **how and when all nodes in a distributed system see the same data after a write operation**.

👉 In simple terms:
> “If I update data, when will everyone else see it?”

⚡ Why Consistency Matters

In microservices + distributed systems:
- Data is spread across services
- Network delays are real
- Failures are normal

So we must define:
👉 How “up-to-date” the system should be

🔐 1. Strong Consistency

🧠 Definition
After a write, **all reads immediately see the latest value**.

⚙️ Behavior
- Write happens
- All nodes instantly reflect update
- No stale reads allowed

📌 Example
Bank balance update:
- You transfer ₹1000
- Immediately every system shows updated balance

⚠️ Trade-offs
- Slower performance

- High coordination cost
- Lower availability in distributed systems

🧠 Mental Model
> “Everyone must agree immediately, even if it slows things down”

⏳ 2. Eventual Consistency

🧠 Definition
System will **become consistent over time**, not immediately.

⚙️ Behavior
- Write happens in one node/service
- Other nodes update later asynchronously
- Temporary inconsistency allowed

📌 Example
E-commerce order system:
- Order placed
- Payment or inventory updates arrive a few seconds later

⚠️ Trade-offs
- Stale reads possible
- Harder to reason about system state
- Better scalability and availability

🧠 Mental Model
> “Everyone will agree eventually, but not right now”

🔗 3. Causal Consistency (Advanced)

🧠 Definition
Operations that are **causally related are seen in the same order by all nodes**.

⚙️ Behavior
- If A causes B → everyone sees A before B
- Independent operations may appear in different order

📌 Example
Social media:
- You post a comment
- Then reply to it
👉 Everyone sees post before reply

⚠️ Trade-offs
- More complex than eventual consistency
- Less strict than strong consistency

🧠 Mental Model
> “Cause must always come before effect”

⚖️ Comparison Table

Model	Speed	Consistency	Availability	Use Case
Strong	Slow	Immediate	Lower	Banking, payments
Eventual	Fast	Delayed	High	E-commerce, social feeds
Causal	Medium	Partial ordering	High	Messaging, collaboration

🏗️ Real-world Mapping (Microservices)

🏦 Strong Consistency
- Payments
- Wallet systems
- Account balance

🛒 Eventual Consistency
- Orders
- Inventory updates
- Notifications

💬 Causal Consistency
- Chat systems
- Comments + replies
- Collaboration tools (Google Docs style)

🔄 Connection to Microservices

Consistency directly affects:

- Saga pattern (uses eventual consistency)
- Event-driven architecture
- Retry + compensation logic
- System design trade-offs

⚠ Interview Trap Insight

If interviewer asks:
> “Why not always use strong consistency?”

👉 Answer:
- It reduces availability
- In distributed systems, network partitions are unavoidable (CAP theorem)

🧠 Mental Model

- Strong = synchronized orchestra 🎻 (everyone plays same note together)
- Eventual = crowd gradually syncing claps 👏
- Causal = conversation flow 💬 (reply must follow message)

🚀 One-line Summary

Consistency models define how quickly and strictly distributed systems agree on data, ranging from strong immediate consistency to flexible eventual consistency optimized for scalability.

CQRS

22 April 2026 19:56

Markdown
🤔 CQRS + Data Duplication (Clean Interview Notes)

🧩 What is CQRS?

CQRS = **Command Query Responsibility Segregation**

It separates a system into two parts:

- 🟤 **Command Model** → Handles writes (create, update, delete)
- 🟤 **Query Model** → Handles reads (fetch/display)

👉 Both models are independent and optimized for their purpose.

⚙️ Core Idea

Instead of one shared model for everything:

- Write side focuses on **correctness + transactions**
- Read side focuses on **speed + query efficiency**

🔄 CQRS Flow

```text
User Request
  ↓
Command (Write Operation)
  ↓
Write Model (Source of Truth DB)
  ↓
Event Published (Kafka / Queue)
  ↓
Read Model Updated (Projection DB)
  ↓
Query Request
  ↓
Read Model Response
📊 Does Data Duplication happen in CQRS?
☑️ Yes — data duplication is expected
CQRS intentionally maintains two copies of data:
Write Model (authoritative data)
Read Model (optimized copy for queries)
👉 This duplication is by design, not a flaw
🧩 Why duplication happens
Write model stores normalized transactional data
```

Read model stores denormalized, query-optimized data
Read models are created as projections of events
⚙️ Example
🟤 Write Model (Source of Truth)
Focus: correctness, consistency, transactions
Plain text
Order Table
- order_id
- customer_id
- status
🟤 Read Model (Query Optimized View)
Focus: fast reads, UI efficiency
Plain text
Order View
- order_id
- customer_name
- product_names
- order_status
👉 This is a duplicated + enriched version of data
🔄 How duplication happens
Command updates Write Model
Write Model emits event (e.g., OrderCreated)
Event is published via Kafka / messaging system
Read Model consumes event
Read DB updates its own projection
Plain text
Write DB → Event → Read DB (Projection / Duplicate Data)
⚖️ Why duplication is acceptable
🚀 Performance
No complex joins required
Faster read responses
🧩 Scalability
Read and write systems scale independently
🎯 Flexibility
Multiple read models can exist for different use cases
⚠️ Trade-offs
❌ Eventual consistency
Read data may lag behind write updates
❌ Storage overhead
Same data stored multiple times
❌ System complexity
Requires event handling + synchronization logic
🧠 Mental Model
🟤 Write Model = official record ledger 📒
🟤 Read Model = optimized display copy 📊
👉 Users never query the ledger directly — they see a shaped version of it.
🔗 Key Interview Insight
CQRS is NOT about avoiding duplication.
👉 It is about:
Accepting controlled duplication to achieve scalability, performance, and flexibility
🚀 One-line Summary
CQRS separates read and write models, and intentionally duplicates data in read models using event-driven projections to achieve high scalability and optimized read performance.

Obserbality

22 April 2026 20:02

Markdown
📊 Topic 5: Observability in Microservices

🧠 What is Observability?

Observability is the ability to **understand what is happening inside a distributed system from the outside**, using logs, metrics, and traces.

👉 If something breaks in microservices:
> Observability tells you **what broke, where it broke, and why it broke**

⚛️ Why Observability is critical

In microservices:
- Many services interact
- Failures are distributed
- Debugging is not straightforward

👉 Without observability:
- You see symptoms ❌
- Not root cause ❌

🧩 The 3 Pillars of Observability

📜 1. Logs

What it is:
- Event-based records of what happened

Example:
```text id="log"
OrderService: OrderCreated for orderId=123
PaymentService: Payment failed due to timeout
Use:
Debugging individual events
Root cause investigation
📊 2. Metrics
What it is:
Numeric data over time
Example:
Request count/sec
Latency (ms)
Error rate (%)
```

Use:
Monitoring system health
Alerting
🔍 3. Traces
What it is:
End-to-end request journey across services
Example:
Plain text
User Request
→ API Gateway
→ Order Service
→ Payment Service
→ Inventory Service
Use:
Identify bottlenecks
Understand service dependencies
🔄 How they work together
Plain text
Trace → shows request path
Metrics → shows system health
Logs → show detailed events inside each step
👉 Together they form full visibility
🏗️ Observability Architecture
Common tools:
Logs → ELK Stack (Elasticsearch, Logstash, Kibana)
Metrics → Prometheus + Grafana
Tracing → OpenTelemetry / Jaeger
⚠️ Why observability is hard in microservices
Too many services
Distributed failures
Network latency variability
Async event flows (Kafka, queues)
🧠 Example (E-commerce checkout failure)
User says:
“Order failed”
Without observability:
You don’t know why ❌
With observability:
Trace shows: Payment Service timeout
Logs show: Gateway timeout after 2s
Metrics show: Payment latency spike
👉 Root cause identified instantly
⚖️ Logs vs Metrics vs Traces
Type. Focus. Use Case
Logs. Events. Debugging
Metrics Numbers Monitoring
Traces. Flow End-to-end analysis
🧠 Mental Model
Logs = diary entries 📖
Metrics = heartbeat monitor 💓
Traces = GPS route map 🗺️
🔗 Connection to other patterns
Observability is critical for:
Circuit breaker tuning
Retry analysis

Rate limiter monitoring
Saga debugging
Event-driven systems troubleshooting

🚀 One-line Summary

Observability is the combination of logs, metrics, and traces that provides full visibility into distributed microservices systems for debugging, monitoring, and performance analysis.

Caching

22 April 2026 20:07

Markdown
📦 Topic 6: Caching Strategies in Microservices

🧠 What is Caching?

Caching is the technique of **storing frequently accessed data in a fast storage layer** so future requests are served faster.

👉 Goal:
> Reduce latency + reduce load on backend systems

⚙️ Why caching is needed

In microservices:
- Databases are slow compared to memory
- High traffic systems need fast responses
- Repeated reads are common (products, profiles, configs)

👉 Without caching:
- High DB load
- Higher latency
- Poor scalability

🧩 Where caching is used

- API Gateway (edge caching)
- Application layer (service cache)
- Database query results
- CDN (static content caching)

🔄 Common Caching Patterns

📦 1. Cache Aside (Lazy Loading)

🧠 How it works:
- Application checks cache first
- If miss → fetch from DB → store in cache

Flow:

```text id="cache-aside"
Request → Cache → (Miss) → DB → Cache updated → Response
☑ Pros:
Simple
Most commonly used
❌ Cons:
First request is slow (cache miss)
📦 2. Write Through Cache
🧠 How it works:
Write goes to cache AND database simultaneously
Flow:
Plain text
Write → Cache → DB (synchronous)
☑ Pros:
Cache always up to date
Strong consistency
❌ Cons:
Slower writes
📦 3. Write Back (Write Behind)
🧠 How it works:
Write goes only to cache first
DB updated later asynchronously
Flow:
Plain text
Write → Cache → Async DB update
☑ Pros:
Very fast writes
❌ Cons:
Risk of data loss if cache fails
📦 4. Cache Refresh / TTL-Based
🧠 How it works:
Data expires after time (TTL)
Fresh data fetched on expiry
Example:
Product details cached for 5 minutes
⚖ Cache Invalidation Problem (BIG INTERVIEW TOPIC)
👉 Hardest problem in caching
Options:
TTL expiration
Event-based invalidation
Manual invalidation on updates
👉 Problem:
“When data changes, how do you ensure cache is updated correctly?”
🧠 Real-world Example
E-commerce Product Page
Product details → cached (high read traffic)
Price updates → invalidate cache
Inventory → real-time or short TTL cache
⚠ Challenges in caching
Cache inconsistency
Stale data
Cache stampede (many requests on expiry)
Memory overhead
Invalidation complexity
🧠 Cache Stampede Problem
```

When cache expires:
Many requests hit DB at same time
👉 Solution:
Locking
Request coalescing
Random TTL jitter
🔗 Cache in Microservices Context
Caching improves:
API Gateway performance
Service response time
Database scalability
🧠 Mental Model
Cache = fast memory desk 🧠
Database = slow library 📚
You don’t go to library every time → you keep notes on desk
🚀 One-line Summary
Caching is a performance optimization technique that stores frequently used data in fast storage layers to reduce latency and backend load, using patterns like cache-aside, write-through, and write-back.

Idempotent

22 April 2026 20:14

📄 IDENTITY: IDEMPOTENCY IN MICROSERVICES
🧠 What is Idempotency?
Idempotency means:
Executing the same operation multiple times produces the same result as executing it once.
👉 In simple terms: Repeated requests should NOT create duplicate side effects.
⚙️ Why Idempotency is Important
In distributed systems:
Network failures happen
Requests get retried
Messages get duplicated (Kafka / queues)
Timeouts cause re-execution
👉 Without idempotency:
Duplicate payments 💸
Duplicate orders 🛒
Data corruption ❌
🧩 Example
❌ Non-idempotent operation
Operation: Add ₹100 to account
If executed 3 times:
₹100 → ₹200 → ₹300 ❌ (wrong in retry scenarios)
☑️ Idempotent operation
Operation: Set balance = ₹1000
No matter how many times executed:
Result = ₹1000 ✔️
🔁 Real-world Microservices Example
Order Service Scenario
❌ Without idempotency:
User clicks “Place Order” twice
Two orders are created
✔️ With idempotency:
Same request is detected using request ID
Only one order is created
🔑 How Idempotency is Implemented
1. Idempotency Key
Client sends unique request ID
Server stores processed request IDs
Flow: Request (ID = 123) → Process → Store result
Request (ID = 123 again) → Return same result
2. Database Constraints
Unique keys (e.g., order_id UNIQUE)
Prevent duplicate inserts
3. State-based Updates
Use absolute updates instead of incremental updates
Example: SET balance = 1000 (not +100)
4. Message Deduplication
Kafka consumer tracks message IDs
Prevents duplicate processing

⚠ Where Idempotency is Critical
Payment systems
Order creation
Inventory updates
Event processing systems
Problems Without Idempotency
Double billing
Duplicate orders
Inventory mismatch
Retry storms causing corruption
Relation with Retry Pattern
Retry + No Idempotency = ❌ Data corruption
Scenario
Result
Retry + Idempotent API
Safe ✔
Retry + Non-idempotent API
Risk of duplication ❌
Mental Model
Idempotent → Light switch (ON → ON → ON = same state)
Non-idempotent → Adding coins (each action changes state)
Connection to Microservices
Idempotency is essential for:
Retry mechanisms
Event-driven systems
Saga pattern
Messaging systems (Kafka / RabbitMQ)
Interview Key Points
Idempotency ensures safe retries in distributed systems
Prevents duplicate side effects
Critical in payments and order systems
Works closely with retry and messaging systems
One-line Summary
Idempotency ensures that repeating the same operation multiple times does not change the result, making distributed systems safe against retries, failures, and duplicate events.
If you want next, I can: 👉 Convert ALL 7 topics into a single “microservices interview PDF booklet”
👉 Or start Topic 8 (Messaging guarantees: Kafka, at-least-once, exactly-once, DLQ)

Deployment models

22 April 2026 20:22

Markdown
🚀 Deployment Models in Microservices (Combined Notes)

🧠 What are Deployment Models?

Deployment models define **how microservices are released, run, and managed in production environments**.

👉 In simple terms:
> “Where services run, how they scale, and how new versions are rolled out.”

🧩 1. Monolithic Deployment (Baseline)

🧠 Definition
Entire application is deployed as a single unit.

⚙️ Characteristics
- One build → one deployment
- Single scaling unit
- Tight coupling

❌ Problems
- No independent deployment
- Hard scaling
- High risk during release

🧩 2. Microservices Deployment (Core Model)

🧠 Definition
Each service is deployed independently.

⚙️ Characteristics
- Independent deployment per service
- Independent scaling
- Separate CI/CD pipelines

☑ Benefits
- Faster releases
- Fault isolation
- Better scalability

☁ 3. Cloud-Based Deployment

🧠 Definition
Microservices deployed on cloud platforms.

⚙ Examples
- AWS (EKS, ECS)
- Azure (AKS)
- Google Cloud (GKE)

☑ Features
- Auto scaling
- High availability
- Managed infrastructure

🐳 4. Container-Based Deployment

🧠 Definition
Services packaged as containers (Docker) and deployed via orchestration tools.

⚙ Stack
- Docker → packaging
- Kubernetes → orchestration

☑ Benefits
- Environment consistency
- Portability
- Easy scaling

🔄 5. Rolling Deployment

🧠 Definition
Gradual replacement of old version with new version.

☸ Flow
- Replace instances one by one
- No downtime

☑ Pros
- Safe deployment
- Continuous availability

❌ Cons
- Mixed versions during rollout

🟢 6. Blue-Green Deployment

🧠 Definition
Two identical environments:
- Blue = current version
- Green = new version

☸ Flow
- Deploy new version in Green
- Switch traffic from Blue → Green

☑ Pros
- Instant rollback
- Zero downtime

❌ Cons
- Higher infrastructure cost

🟡 7. Canary Deployment

🧠 Definition
New version is released to a small subset of users first.

☸ Flow
- 5% traffic → new version
- Monitor metrics
- Gradually increase traffic

☑ Pros
- Very low risk
- Real user validation

❌ Cons
- Requires strong observability

🌍 8. Multi-Region Deployment

🧠 Definition
Services deployed across multiple geographic regions.

⚙ Purpose
- Low latency
- High availability
- Disaster recovery

📌 Example
- US region
- Europe region
- Asia region

🔁 9. Hybrid Deployment

🧠 Definition
Combination of:
- On-prem systems
- Cloud systems

⚙ Use case
- Legacy + modern systems coexist

🌐 10. Active-Passive Deployment

🧠 Definition
Only one environment actively serves traffic, others are standby.

⚙ Flow
```text id="active-passive"
```

Active Region → Handles all traffic
Passive Region → Standby (failover backup)
☑ Pros
Simple design
Easy consistency management
❌ Cons
Underutilized resources
Slower failover
🟢 11. Active-Active Deployment
🤔 Definition
Multiple environments actively serve traffic at the same time.
⚙️ Flow
Plain text
Region A → part of traffic
Region B → part of traffic
Region C → part of traffic
☑ Pros
High availability
Better performance
Fast failover
❌ Cons
Data consistency complexity
Conflict resolution challenges
⚖️ Deployment Model Comparison
Model
Downtime
Risk
Cost
Complexity
Use Case
Monolithic
High
High
Low
Low
Legacy systems
Microservices
None
Medium
Medium
High
Modern systems
Rolling
None
Medium
Medium
Medium
Standard deployments
Blue-Green
None
Low
High
Medium
Critical apps
Canary
None

Very Low
Medium
High
Large-scale systems
Multi-Region
None
Very Low
Very High
Very High
Global systems
Active-Passive
Minimal
Low
Medium
Low
DR systems
Active-Active
None
Low
High
Very High
Global high-scale apps
🧠 Mental Model
Rolling = replacing parts while machine runs 🔧
Blue-Green = switching between two factories 🏭
Canary = testing with small group first 🐤
Active-Passive = backup standby system 🪑
Active-Active = multiple live systems sharing load 🌍
🔗 Connection to Microservices
Deployment models directly affect:
Service discovery 🔎
Observability 📊
Resilience patterns ⚡
Saga workflows 📜
Scaling strategies 📈
💡 Interview Key Points
“Microservices enable independent deployment per service.”
“Blue-green and canary reduce production risk.”
“Active-active improves availability but increases complexity.”
“Deployment strategy depends on cost, scale, and consistency needs.”
🚀 One-line Summary
Deployment models define how microservices are released and operated in production, ranging from rolling updates to canary, blue-green, and multi-region active-active/passive strategies for scalability and reliability.